AI Engineering and Claude Architect · Part II, Retrieval

Vector Search, Lesson 2

ANN Indexes in Production: Filtering and Quantization

The tenant filter passes 10% of rows, and the formula says almost no query should come back empty. In production, 88% do. The formula assumed filters ignore the vectors, and tenants don't. This lesson measures what the textbook version leaves out.

Module	AI04 of 13
Lesson	2 of 5
Prerequisites	AI04 L1, HLD M21A (IVF, PQ, HNSW, the post-filter cliff)
Exam relevance	Out of scope
Reading time	~35 minutes

Contents

1. ANN Indexes in Production: Filtering and Quantization

Lesson 2 of 5 · Vector Databases and Search

ANN Indexes in Production: Filtering and Quantization

Key idea

Index quality isn't a fixed property. It depends on your data, your settings and your filters, so recall has to be measured continuously. Filters correlated with the vectors, such as tenants, behave far worse than the textbook estimate. **Quantization can shrink memory by 4 to 32 times, but how much recall it costs varies wildly with the data**, so test it before you trust it, and keep the original vectors for reranking.

> *Before: HLD M21A teaches how IVF, PQ and HNSW work, and derives the post-filter cliff for independent filters. This lesson builds on them and doesn't repeat them. Teaches: measuring recall, tuning knobs, correlated filters, iterative scans, scalar and binary quantization, index operations. Used by: AI04 L4–L5, AI05 L7.*

The hook: the filter that broke the formula

HLD M21A derived that with a pass rate `p`, the chance that all `n` candidates fail is `(1 - p)ⁿ`. With 40 candidates and a 10% filter, that's about 1.5%, so almost no query comes back empty.

Real output from `code/ai04-vector/VectorLab.java`, using 20,000 synthetic vectors in 40 clusters, 200 queries, and 40 candidates:

```
Filter independent of the vectors, 40 candidates, want 10
  pass=0.50  avg matches= 20.5 empty queries=  0.0% scan needed for 10=      19
  pass=0.10  avg matches=  3.9 empty queries=  0.5% scan needed for 10=      99
  pass=0.01  avg matches=  0.5 empty queries= 61.5% scan needed for 10=   1,120
Filter tied to the vectors: a tenant owning 4 of 40 clusters
  pass=0.10  avg matches=  4.8 empty queries= 88.0% scan needed for 10=   1,405
```

The 10% row matches pgvector's documentation, which gives an example of about 4 matches on average with HNSW and an `ef_search` of 40. So the simulation is in the right range for the independent case. Now look at the last row.

> *Key idea*
>
> **Same 10% pass rate. Similar average of matches. But 88% of queries return nothing.** The average hides it. When a query lands near the tenant's clusters, nearly all candidates match. When it lands elsewhere, none do. So the tenant filter isn't the independent coin flip that the formula assumes.

This isn't an unusual case. Tenants, languages, product categories and date ranges are **all correlated with content**. HLD's cliff is the best case.

Intuition: searching a library by shelf

You're looking for books near a topic, restricted to books published by one small publisher.

If that publisher's books are scattered evenly, then checking the 40 nearest books finds a few. But if they only publish cooking books, and you're searching for physics, then the 40 nearest physics books include none of theirs, however many they've published. You have to walk a very long way, or go straight to the publisher's shelf.

That shelf is a partition, and it's why filtering strategy matters so much for tenants.

Measure first: recall is your production metric

Recall@k is the share of the true top `k` that an approximate search returns, as HLD M21A defined it. pgvector's documentation is direct: "Monitor recall by comparing results from approximate search with exact search."

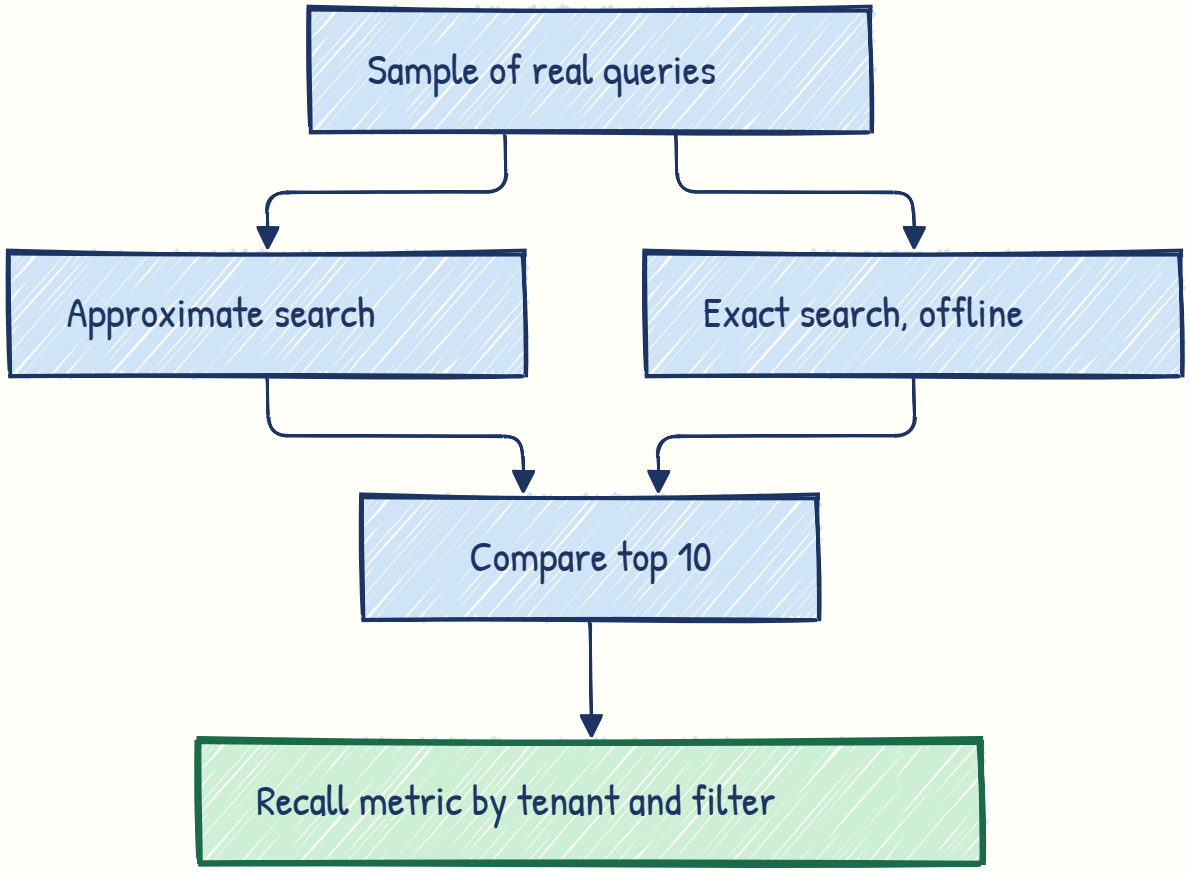


Figure 1. Continuous recall measurement. A sample of real queries runs both ways. Exact search is slow, so it runs in the background.

In pgvector, exact search is available in a transaction using `SET LOCAL enable_indexscan = off`. **Segment the metric**, as AI01 L2 taught: overall recall can hide a tenant or a filter that's broken.

Tuning knobs, by what they cost

Index	Knob	pgvector default	Raises	Costs	Can you change it live?
HNSW	`m`	16	Recall	Memory, build time	Rebuild
HNSW	`ef_construction`	64	Recall	Build time	Rebuild
HNSW	`hnsw.ef_search`	40	Recall	Query latency	**Yes, per query**
IVFFlat	`lists`	Suggested `rows/1000` up to 1M, then `sqrt(rows)`	Speed	Recall, if probes stay low	Rebuild
IVFFlat	`ivfflat.probes`	1, with `sqrt(lists)` suggested as a start	Recall	Query latency	**Yes, per query**

What `probes` buys

```
IVF with 141 lists over 20,000 vectors
  nprobe=1   recall@10=0.491 vectors scanned=  1.0%
  nprobe=2   recall@10=0.767 vectors scanned=  1.8%
  nprobe=5   recall@10=0.998 vectors scanned=  3.8%
  nprobe=10  recall@10=1.000 vectors scanned=  7.4%
  nprobe=20  recall@10=1.000 vectors scanned= 14.6%
```

Two lessons, with the caveat that this synthetic data is cleanly clustered, so real data will need more:

1. **The default of 1 probe gave under half the true neighbors here**, with no warning. Defaults are starting points.
2. **The curve flattens quickly.** Going from 5 to 20 probes costs almost four times the work, and adds almost nothing. Find the knee on your own data.

Filtering, properly

HLD M21A names three strategies. Here's what they mean in a real product, using pgvector.

What actually happens

pgvector's documentation says approximate indexes apply filters **after** the index scan. So a plain `WHERE tenant_id = ?` with HNSW is post-filtering, with the failure above.

Approach	How	Best when	Weakness
B-tree on the filter, exact search	The planner filters first, then sorts by distance	The filter is very selective	Slow when many rows match
Iterative index scan	Keeps scanning the index until enough rows match	Moderately selective filters	Bounded by a scan limit, so it can still come up short
Partial index	`WHERE category_id = 123` on the index	A few distinct values	One index per value
Partitioning	Separate partitions, each with its own index	Many distinct values	Operational complexity
Filtered ANN	The engine walks the graph with the filter	Dedicated engines	Depends on the engine

```
-- Iterative scan: keep going until 10 rows pass, up to the limit.
BEGIN;
SET LOCAL hnsw.ef_search = 100;
SET LOCAL hnsw.iterative_scan = 'relaxed_order';
SELECT id FROM chunks
WHERE tenant_id = 'acme'
ORDER BY embedding <=> $1
LIMIT 10;
COMMIT;
```

These settings are from pgvector's documentation. `relaxed_order` gives better recall, and results may be slightly out of order, while `strict_order` keeps exact distance order. `hnsw.max_scan_tuples` caps the walk, at 20,000 by default.

Compare that with the lab: finding 10 matches for the correlated filter needed about 1,405 steps on average, on a mere 20,000 vectors. On a far larger index, a rare tenant can hit the scan limit and return too few results. **So iterative scans help with filters, while they aren't a tenant isolation strategy on their own.** AI04 L5 chooses one.

Choosing a strategy

Filter shape	Recommendation
Passes more than half of rows	Plain post-filter, with some over-fetch
Passes around 1–50%, independent of content	Iterative scan or filtered ANN
Very selective, under about 1%	Exact search on a filtered subset
Tenant, or correlated with content	Partition or namespace by it
A few fixed categories	Partial indexes

This table is an engineering recommendation. Thresholds depend on your data, so verify them using your recall metric.

Quantization: paying less for memory

HLD M21A covered product quantization. Two simpler methods are now common, and pgvector supports related options.

Method	Idea	Size vs float32	pgvector route
Half precision	16-bit floats	50%	`halfvec`, indexed up to 4,000 dimensions
Scalar (int8)	One byte per dimension	25%	Through other engines, or `output_dtype` from Voyage
Binary	One bit per dimension, its sign	3.1%	`binary_quantize`, using Hamming distance

Real measurements on the same synthetic data:

Method	Memory	recall@10
float32, exact	100%	1.000
int8 scalar quantization	25.0%	0.938
binary, Hamming only	3.1%	0.125
binary top 10 then exact rerank	*	0.125
binary top 40 then exact rerank	*	0.341
binary top 100 then exact rerank	*	0.577

* the rerank step also needs the float vectors, usually from disk

This result surprised me, so read it carefully

Binary quantization did **badly** here. It's often reported as working well, and pgvector documents a pattern for it. **So why the poor result? This data is tightly clustered: points in a cluster mostly share the same signs, so one bit per dimension can't separate near neighbors.** Real embeddings may do much better, and some models are trained for binary output.

The right lesson isn't "binary is bad". It's that a technique's recall depends on your data, so measure before you trust either a blog or this page.

The rerank pattern

Keep compact vectors in RAM for a fast first pass, and full vectors on disk for a final ordering. pgvector documents this with a binary index, re-ranking the top candidates by the original vectors.

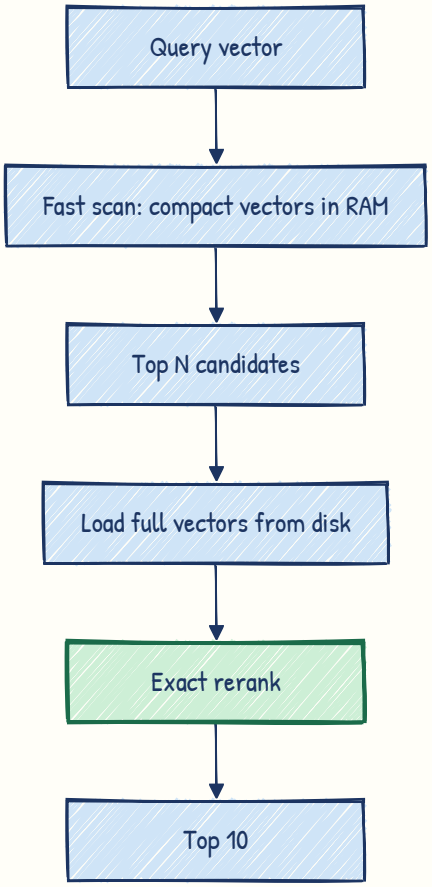


Figure 2. Two-stage search. A compact index finds candidates cheaply, then full-precision vectors decide the final order.

The size of `N` is a recall knob. In the lab, widening it from 10 to 100 lifted recall from 0.125 to 0.577. The first stage's quality determines how large `N` must be.

Operating an index

Concern	What to know
Build time	pgvector builds faster if the graph fits in `maintenance_work_mem`, and gives a NOTICE when it doesn't. Don't exhaust server memory
Parallel builds	`max_parallel_maintenance_workers` speeds up builds
IVFFlat timing	Create it after the table has data, since the clusters are fitted then
Deletes	HNSW deletes are tricky (HLD M21A), so expect dead entries and periodic maintenance
Drift	IVF centroids drift as data changes. Recall falls silently
Memory pressure	An HNSW index that spills out of RAM gets much slower. Size it so that it fits
Dimension limits	pgvector indexes `vector` up to 2,000 dimensions, `halfvec` to 4,000, and `bit` to 64,000

How the field approached it

Problem	Approach	Source
Exact search doesn't scale	HNSW, IVF, PQ	HNSW paper, HLD M21A
Filtering after ANN returns too little	Iterative scans, filtered graph walks	pgvector docs
Memory cost	Scalar and binary quantization, with rerank	Same
Silent decay	Comparing against exact search	Same

Anthropic isn't involved at this layer. It doesn't sell a vector database, so this part of your stack is your own choice.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: one index, default settings. It looks fine in a demo, with default probes and post-filtering.

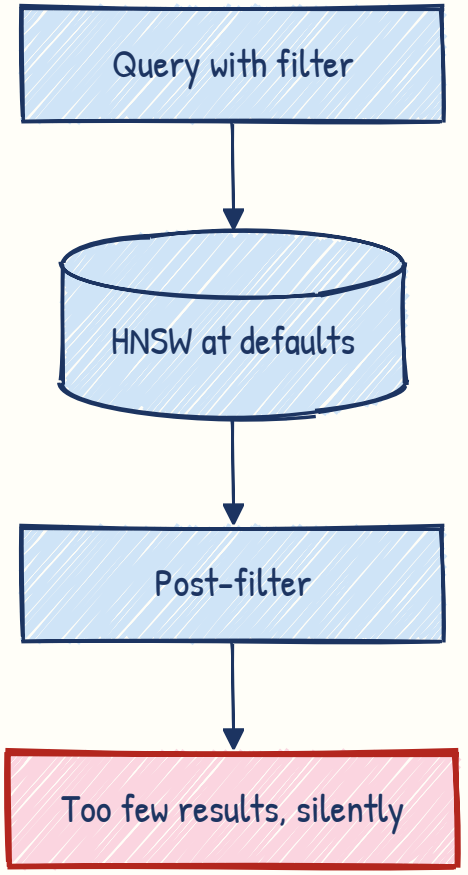


Figure 3. Baseline. Defaults and post-filtering, with nothing measuring recall, so failures stay invisible.

Improved: tune using a recall metric. Quality is now visible, and improves. New problem: higher `ef_search` and larger RAM use get expensive at scale, and rare filters still fall short.

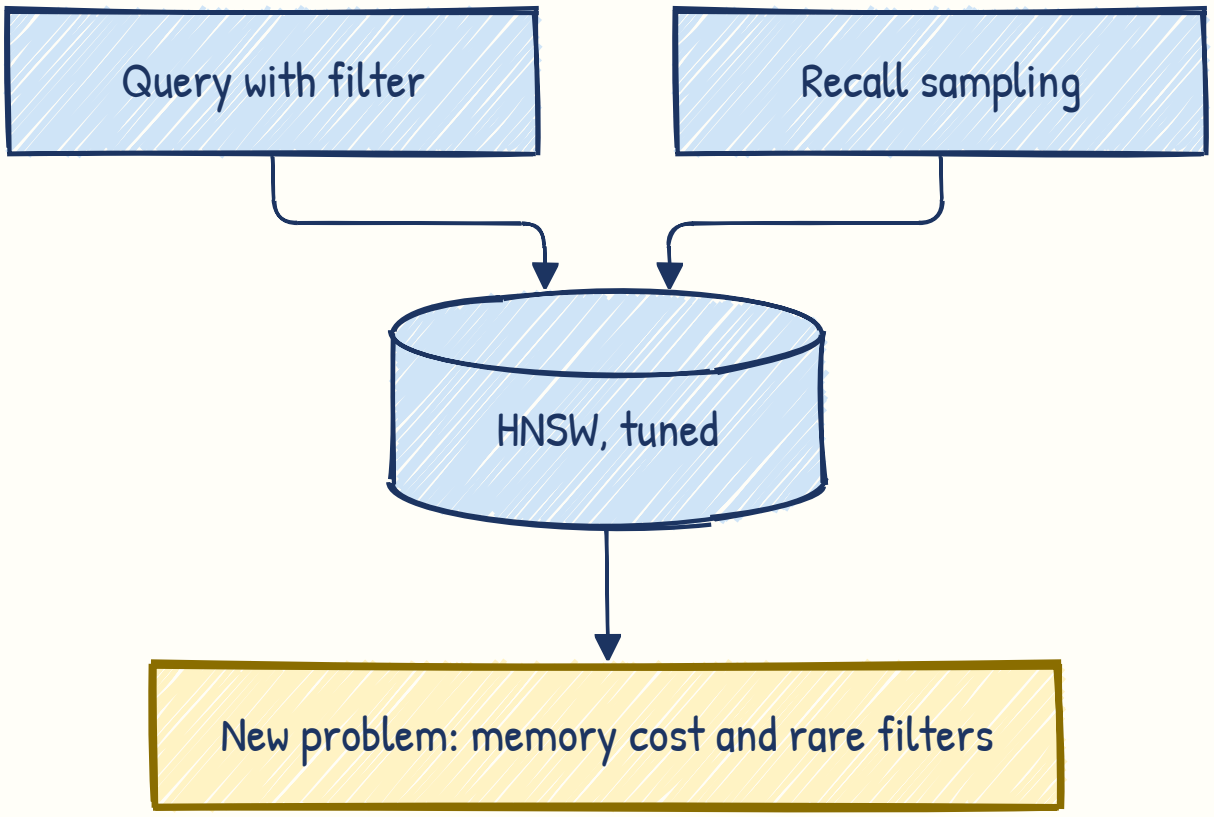


Figure 4. Improved. Measured recall guides tuning, but pure tuning eventually hits cost limits.

Production: route by filter shape, with two-stage search.

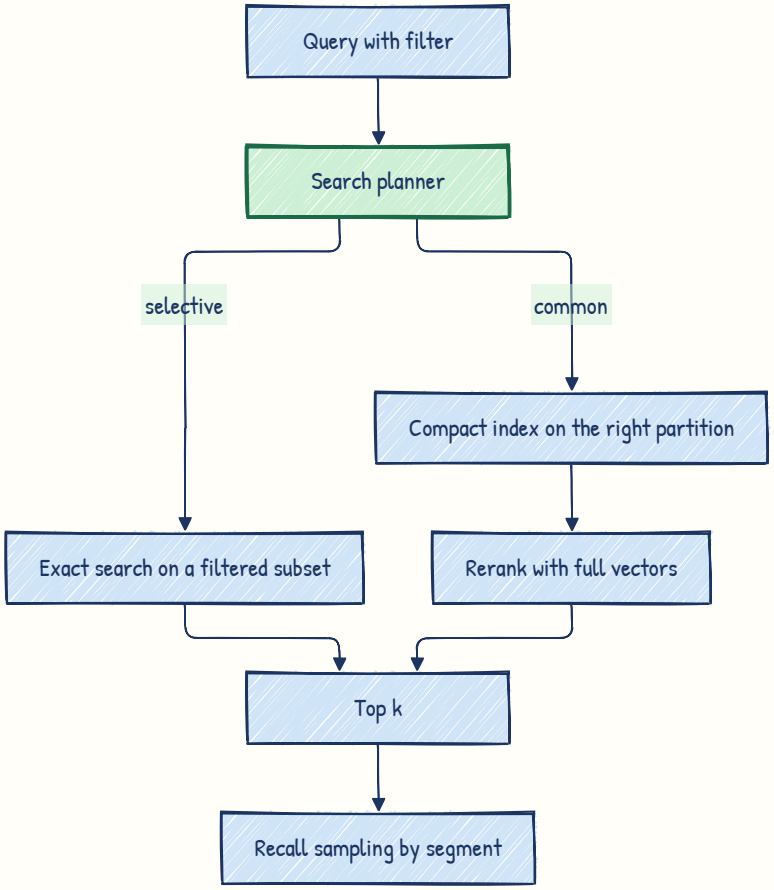


Figure 5. Production. A planner chooses the path by filter shape. Compact vectors serve the first stage, and full vectors in storage decide the final order.

Component	Why it exists	How it communicates
Search planner	Picks the path, using table statistics	In-process, or the database's planner
Compact index	Keeps memory affordable	ANN engine
Full vector store	Final ordering, and re-quantization	Disk or object storage
Recall sampling	The only way to see silent decay	Background job against exact search

Failures

Failure	Containment
Index grows beyond RAM	Alert on memory, then quantize, partition or shard
Recall decays with drift	Sampling alerts, then rebuild
A new rare filter arrives	Segment metrics catch it
Long index rebuild	Build beside the old index, and swap
Too few results	Fall back to exact search, or return fewer with a flag

Trade-offs

Decision	Gain	Cost
Higher `ef_search`	Better recall	Latency
Iterative scan	Fuller filtered results	Variable latency
Partitioning by tenant	Reliable isolation and filtering	Many indexes to operate
Scalar quantization	4× less memory	Some recall, 0.938 in the lab
Binary quantization	32× less memory	Highly data-dependent recall
Two-stage rerank	Recovers precision	Extra I/O and latency

When to use no index at all: for thousands of vectors, or very selective filters, exact search is simple and perfect.

Common mistakes

1. **Trusting default settings.** One probe found under half the true neighbors in the lab.
2. **Assuming a filter is independent of the vectors.** Tenants aren't.
3. **Reading average matches only.** The 88% empty rate was hidden by an average of 4.8.
4. **Adopting quantization from a blog post.** Measure on your own data.
5. **Discarding full vectors after quantizing.** You lose reranking and re-quantization.
6. **Monitoring latency only.** Recall decays with no error.

What to say in an interview

"I'd measure recall continuously against exact search, by segment. For filters, I'd check correlation, not just selectivity: a 10% tenant filter behaved far worse than a random 10% in my test. So tenants get partitions or namespaces, while moderate filters use iterative scans or filtered ANN. I'd only adopt quantization after measuring it, and I'd keep full vectors for reranking."

Certification lens

How this lesson maps to CCAR-F

Not tested. Vector database implementation details are out of scope. One general habit does carry over: an aggregate can hide a broken segment (TS 5.5).

These are original practice questions for understanding, not exam simulations.

Q Q1 · Troubleshooting · Hard. A tenant filter passes 8% of rows, yet many queries return nothing, even though `(0.92)^40` suggests only 3.6% should. Why might the formula be wrong, and what would you change?

A The formula assumes the filter is independent of the vectors. Tenants cluster by topic, so a query can land far from all of that tenant's data, giving zero matches. Partition or namespace by tenant, so the search only walks that tenant's data. Iterative scans help, but can hit their limit.

Q Q2 · Configuration · Medium. In pgvector, you want better recall on one query without rebuilding the index. Which settings could you change?

A `SET LOCAL hnsw.ef_search` for HNSW, or `ivfflat.probes` for IVFFlat, and optionally `hnsw.iterative_scan` for filtered queries. `m`, `ef_construction` and `lists` need a rebuild.

Q Q3 · Best decision · Hard. A team reads that binary quantization saves 97% of memory, and proposes discarding full vectors. What do you recommend?

A Measure recall on your own data first, since it's highly data-dependent, and it was poor in this lesson's lab. Keep full vectors on disk regardless, so you can rerank, re-quantize and migrate. Compare with half precision or scalar quantization, which cost less recall in many cases, and decide on the measured curve.

Revision - Lesson 2

→ Recall isn't fixed. Measure it against exact search on sampled queries, by segment.

→ HNSW: m and ef_construction need a rebuild. ef_search changes per query.

→ IVF: one probe found under half the neighbors in the lab, and the curve flattens quickly.

→ pgvector applies filters after the index scan. A 10% filter leaves about 4 of 40 candidates.

→ Correlated filters are worse: 88% of queries came back empty at a 10% pass rate.

→ Averages hide this, so watch the empty-result rate.

→ Options: exact search for selective filters, iterative scans, partial indexes, partitions, filtered ANN.

→ Quantization: half precision 50%, int8 25%, binary 3.1% of size.

→ Recall is data-dependent: int8 0.938, binary 0.125 on the lab's clustered data.

→ Keep full vectors, and rerank in a second stage.

Your notes